Lab06 - LSP & Formatting

Computer Language Processing 2025 Extension Lab Final Report

Contact Information

- Lilian Noé
  - email: lilian.noe@epfl.ch
- Roman Batut
  - email: roman.batut@epfl.ch
- Tristan Cendan
  - email: tristan.cendan@epfl.ch

Abstract

The Lab06 was the final lab of the whole semester's project. The five first labs correctly and fully implement a compiler for Amy, and the sixth lab introduces the base of a language server and a formatter for this language. The motivation behind the choice of these features is the need for a guided and familiar user experience. In theory, we would want the formatter to turn any tokenizable .amy file into a well structured and standard form. This would imply removing unnecessary white spaces and new lines, arrange parentheses and brackets, comments, etc... The language server should work seemlessly with VSCode, giving access to code highlighting and goto definitions, which means navigating between Amy files by clicking on variables defined in another file.

Introduction

In the CS-320 class, Computer Language Processing, we implemented a compiler from a language called Amy to WebAssembly. Amy is similar to Scala, but a lot simpler. The exact specifications can be found here.

The compiler is built as a pipeline, i.e. a succession of steps that gradually manipulate and transform the input. We start with a raw text file, or even multiples if there are dependencies. This string is then tokenized by the lexer, before being abstracted into an Abstract Syntax Tree by the parser, and finally turned into WebAssembly. Between the last two steps, the compiler takes care of the different variables, functions, classes and types name, as well as the types themselves, thanks to the name analyzer and the type checker. Those steps specifically determine whether the program can be compiled.

Our extensions are a formatter and a language server. A formatter is quite self explanatory ; it allows to have a consistent and familiar syntax. The language server allows for more in depth integration of the language when using an IDE, providing highlighting, goto definitions, etc... We believe those extensions fit well in this whole project, as they focus on the user experience, a point that was not considered before. Those tools would allow for a more intuitive and familiar environment to the programmer.

The formatter is simply a fork at a set point of the pipeline, reusing the already present infrastructure and only adding components when absolutely needed. The language server acts as a standalone. It uses the whole compiling pipeline, then wraps around it with the necessary server and client infrastructures.

More precisely, the LSP (Language Server Protocol) implementation is split into a Client, on the IDE side, and a Server on the language side. The Server is the part that needs a compiler to be implemented. It may use any part of the language implementation, but regarding the go-to-definition feature, it uses only a part of the actual pipeline which is namely the Lexer, the Parser and the Name Analyser. Indeed, the Server receives text files from the LSP client and, in order to find the definition of a variable, we must obtain identifiers so that we can compare them. Our goal is not, for go-to-definiton at least, to generate executable code or even to check the validity of the program for the moment, that's why we don't follow the pipeline up to the end.

Examples and Use of the Extension

Formatter

Let's suppose we have the following unclear and confusing module:

```
        object Hanoi

def

solve(n : Int(
    32)
) : Int(32) = {
     if (n < 1) {      error("can't solve Hanoi for less than 1 plate")
   }                else
   {      if (n == 1) {        1      } else {
     2 * (solve(n - 1) + 1)      }
 }  }  Std.printString("Hanoi for 4 plates: " ++ Std.


intToString(solve(4)
))

end                 Hanoi
```

Thankfully, the Amy compiler can successfully tokenize this code. Since this condition is met, running the Amy compiler with the `--format` parameter will turn this unusable piece of code into this consistent and standard format:

```
object Hanoi

  def solve(n: Int(32)): Int(32) =
  {
    if (n < 1)
    {
      error("can't solve Hanoi for less than 1 plate")
    }
    else
    {
      if (n == 1)
```

```
        {
          1
        }
        else
        {
          2 * (solve(n - 1) + 1)
        }
      }
    }

    Std.printString("Hanoi for 4 plates: " ++ Std.intToString(solve(4)))
  end Hanoi
```

This formatted string will then be written back to its corresponding file. The formatter does not care about the number of files or the number of modules in each file. Similarly, it does not care about the content of the module itself. The only condition that implies the possibility of formatting is that the code reaches successfully the end of the lexer.

The comments are handled correctly as well. Single line comments are printed on their own lines. The following lines

```
Std.printInt(gcd(17, 99)); // 1
Std.printInt(gcd(16, 46)); // 2
Std.printInt(gcd(222, 888)) // 222
```

are turned into

```
Std.printInt(gcd(17, 99));
// 1
Std.printInt(gcd(16, 46));
// 2
Std.printInt(gcd(222, 888))
// 222
```

The same behaviour can be observed with multiline comments, which remain untouched.

Language Server

The language server provides support for the code highlighting as follows, directly in the IDE

```
object Hanoi

  def solve(n : Int(32)) : Int(32) = {
    if (n < 1) {
      error("can't solve Hanoi for less than 1 plate")
    } else {
      if (n == 1) {
        1
      } else {
        2 * solve(n - 1) + 1
      }
    }
  }

  Std.printString("Hanoi for 4 plates: " ++ Std.intToString(solve(4)))
end Hanoi
```

The Language Server also implements the Go-to-definition feature, which consists in being redirected to the definition of a variable or a function when clicking on an occurrence of it. A user should only have to Ctrl+Click (or Cmd+Click on Mac) on a variable, or press F12 when the variable is selected. Then the IDE detects the user's request, asks for the definition and the Client side of the LSP asks the Server to provide the definition position. The following screenshots illustrate how it works on VSCode with the extension enabled.

```
object Hanoi

  def solve(n : Int(32)) : Int(32) = {
    if (n < 1) {
      error("can't solve Hanoi for less than 1 plate")
    } else {
      if (n == 1) {
        1
      } else {
        2 * solve(n - 1) + 1
      }
    }
  }

  Std.printString("Hanoi for 4 plates: " ++ Std.intToString(solve(4)))
end Hanoi
```

```
/** This module contains basic functionality for Amy,
  * including stub implementations for some built-in functions
  * (implemented in WASM or JavaScript)
  */
object Std
  def printInt(i: Int(32)): Unit = {
    error("") // Stub implementation
  }
  def printString(s: String): Unit = {      Lil-Noe, 3 days ago • feat:
    error("") // Stub implementation
  }
  def printBoolean(b: Boolean): Unit = {
    printString(booleanToString(b))
  }

  def readString(): String = {
```

Implementation

Background

For the formatter, the given suggestion from the teaching team mentioned that it could be done at two levels ; either the lexer or the parser. Intuitively, we went for the parser level, as it seemed easier to format. With this in mind, we started modifying the pipeline to fork the pipeline after the original parser, and then to pretty-print the generated AST in the files. We understood fast that the already existing infrastructure wouldn't be sufficient to perform the whole operation.

For the following steps, we followed our instinct backed by ChatGPT suggestions. We decided to create an alternate formatting parser, that would include everything that we need. It worked as expected, allowing for

case class arguments to be printed for example, while using a common parser trait. To print, we used the Document and Printer traits already present in the project.

When we finally came to adding the comments, we realised this way of doing was simply not robust enough to implement them in a nice way. The fact that comments could occur anywhere, including inside AST nodes, made it impossible to continue like without requiring hacky workarounds.

Because of this issue, we looked at the different other options. We settled on working at the lexer level. The idea is simply to iterate on the tokens while keeping an indenting amount in memory, inserting white spaces or newline characters when deemed necessary. Finally, the generated string overwrites the file that we want to format.

As for the language server, we followed a more theorical and academic background. We used a paper from Malintha Ranasinghe, “A practical guide for LSP”, as well as the Microsoft specification for LSP, and the Language Server extension guide from VS Code. The idea is to communicate between a server corresponding to the language, and a client associated to an editor. The idea for LSP is that an Amy Language Server should be able to work with every IDE that implements a Language Client. This means we had to learn how VSCode deals with LSP to create an extension that can implement it. On the other hand we also had to learn about how a Language Server can be implemented, which informations it receives from a client and how to deal with it.

Regarding the Go-To-Definition, the conceptual idea is to receive the position and the file in which the user had clicked from the client side. Then to process it using the pipeline, going through the program by comparing positions to find the identifier. When it's done, we go through the program once more to compare identifiers and find its definition. And finally we need pass the file and position of this definition to the client.

Finally, the code highlighting required the creation of a VSCode extension. This is quite straightforward, as the VSCode documentation provides a lot of information on how to create an extension. The idea is to use the TextMate grammar to highlight the code, which is a common way to do it in VSCode.

Implementation details

Concerning the formatter, we stumbled upon a few hard parts. The first one was simply keeping track of which file each module corresponded to. Indeed, the lexer removes the concept of files, and simply arranges appends all the modules of all files.

To fix this, instead of running the lexer on all files as in the usual pipeline, we extracted the lexing process and used to run to tokenize each file, one by one. Thanks to this small trick, we can still run the full pipeline as exptected when compiling, and keep track of the files when formatting.

Then, it came down to the printing. We use a `StringBuilder` to easily append to the already existing string. Since we simply used a `match` expression to switch over the tokens, we couldn't use the already existing tools provided by Scallion and Sylex in this step.

It crossed our mind to use the Scallion library at this step, as it's very close what we want. However, due to the comments and the rigidity of the library, we found it easier to mimick the behaviour of Scallion's Parsers, in an homemade function.

One issue that we faced at this step was the different formatting we had to make for the same tokens, depending on the context. For example, when faced with an identifier, it could make sense to skip a line after it, or skip a line before it, or to not skip a line at all.

To address this issue, we introduced a new variable that keeps track of the last encountered token before the current one. This makes it a lot easier to know what to do in almost every situations.

More conceptually, it also was a struggle to know where to add the different kind of special characters. For example, it does make sense to have white spaces around an operation token, but should those white spaces be introduced when considering the operation token, or when considering its operand ? Having a reliable and robust version of this took a lot of time and trial & error.

The language server was a bit more straightforward, but still required a lot of work. To launch it, we created a new program parameter `--server_mode` that simply needs to be specified when running the program. For the implementation itself, the first step was to create the server and client sides, which we did using the LSP4J library. This library is a Java implementation of the Language Server Protocol, and it allows us to create a server that can communicate with a client, such as VSCode.

Concerning the Go-to-definition feature, the first struggle was to manage the paths. When the server receives a URI from the client indicating where the user clicked, it often needs access to definitions from other files to resolve references correctly. To address this, we first navigate back to the root directory where the client was launched. From there, we include all .amy files in the compilation process to ensure that all required modules are available.

The second part was to determine wether an identifier could be found in a specific Module, and hence go look for it recursively. To begin, the Amy position file starts at (1, 1) where the LSP4J gives a position that starts at (0, 0). So we had to translate it in the first place. When we finally managed to find an identifier, we must be sure it has a range position to compare it with the given position. In our original pipeline, the parser would give only one position that corresponded to the beginning of the identifier in it. The problem is we had to get the start and end position of a given identifier to be sure the client clicked on it. Hence we modified the Position file and the parser to include start and end positions of the identifier to the nodes.

The most difficult part of this goal was the Match tree, because we had to look in every match case. In them, we had to look in the scrutinees and in the expressions only if it wasn't in the scrutinee. The thing is the scrutinee may be another case class, so we had to go once again recursive using another function to make sure we had covered all the possiblities in which there could be an identifier.

Finally, once we had the identifier we had to look in the program for the definition of it. Once again, only specific nodes can include definitions, in which case we would have to go recursive. The thing was only to compare identifiers so it was easier than finding the identifier. Then we built the message to send to the client, using once again the translation library, and setting the position to the end of the definition instead of the beginning.

Possible Extensions

As of the current state of our extensions, the formatter seems to be working as expected in the majority of the cases. We incorporated a few basic tests that should cover a large range of the possibilites, and with the manual testings we did, no unexpected result was found.

There are a few tests on the LSP server side that make sure the go-to-definition works as expected for different files and identifiers, either variables or functions. There is even a test that makes sure the server is well launched. The language server only performs one feature, which is go-to-definition. But as we already made a part of the job by creating the client and server sides and learning how to launch it and use it, it should be quite easy to implement other features of the LSP, especially the ones that use the same functionality as the go-to-definition, such as hover feature.

In theory, there are a lot more things a language server can offer, going from auto completion to error checking. We believe these specific features would require a very different apporach and a lot of work, especially for auto-completion if it needs to generate code that is not already finished.

In its globality, we found this project very fun and educating. We believe we've got a deep and global understanding of the compiling pipeline, and we would like to thank the teaching team for this wonderful semester.